

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación

Carrera de Software
Asignatura: Aplicaciones Web

BIOPET

Sistema web de gestión veterinaria

Informe final de la Tercera Entrega v0.9.0-rc

Equipo **BMT**

Mariscal Cabrera Jaime Josué
Beltrán Montiel Fred Adrián
Taipe Mora Zaida Melissa

Docente: Guerrero Ulloa Gleiston Ciceron

Período académico: 2025 – 2026 — Curso/paralelo: A

Quevedo, 31 de julio de 2026

Índice general

1. Resumen ejecutivo	1
2. Estado actual del sistema	4
2.1. Cierre de observaciones de las Entregas 1A y 1B (Bloque 0)	4
2.2. Auditoría OWASP (resumen)	4
2.3. Versión y alcance	5
2.4. Frontend	5
2.5. Backend	5
2.6. PostgreSQL	6
2.7. Redis	6
2.8. Docker Compose	6
2.9. HTTPS/TLS	6
2.10. Autenticación	6
2.11. Mascotas	6
2.12. Cobertura	7
2.13. Rendimiento	7
2.14. Documentación	7
2.15. Estado de las pruebas	7
3. Arquitectura C4	8
3.1. C4 Nivel 2 — Contenedores	8
3.2. C4 Nivel 3 — Componentes del backend	8
3.3. Decisiones de arquitectura relevantes (ADR)	9
4. Matriz de trazabilidad	10
5. Protocolo experimental	16
5.1. Entorno de ejecución	16
5.2. Comandos reproducibles	16
5.3. Mediciones de rendimiento y caché (Fred)	17
5.4. Mediciones de seguridad (Jaime)	17
5.5. Cobertura de pruebas (Jaime)	17
5.6. Usabilidad (SUS) y accesibilidad (Lighthouse) — Zaida	18
5.7. Diferenciación de los tipos de medición	18
6. Resultados — Jaime (seguridad y backend)	19
6.1. Autenticación JWT por cookies	19
6.1.1. Logout y revocación en Redis	19
6.2. Autorización por rol y por propietario	20

6.3. Rate limiting de login	20
6.4. <code>ProblemDetail</code> uniforme	20
6.5. Auditoría A09	20
6.6. Cabeceras de seguridad y TLS	21
6.7. Protección estructural contra inyección (A03)	22
6.8. Postman	22
6.9. Cobertura JaCoCo y resultado de pruebas	22
6.10. C4 Nivel 3 y ADR-006	23
6.11. Limitaciones reales de este bloque	23
7. Resultados — Fred (base de datos, reproducibilidad y rendimiento)	25
7.1. Base de datos y PostgreSQL	25
7.2. Función de resumen y procedimientos almacenados	25
7.3. Reproducibilidad y Docker	26
7.4. Redis y caché	26
7.5. Rendimiento (k6)	26
7.6. Análisis estadístico	27
7.7. Limitaciones registradas	27
8. Resultados — Zaida (requisitos, frontend y experiencia de usuario)	28
8.1. Requisitos	28
8.2. Frontend Angular	28
8.3. Autenticación mediante cookies (integración frontend)	29
8.4. CRUD accesible de mascotas y accesibilidad manual	29
8.5. Usabilidad (SUS)	29
8.6. Accesibilidad automática (Lighthouse)	29
8.7. Publicabilidad y documentación funcional	30
8.8. Limitaciones reales de este bloque	30
9. Amenazas a la validez	31
9.1. Validez interna	31
9.2. Validez externa	31
9.3. Validez de constructo	32
9.4. Validez de conclusión	32
10. Ética y protección de datos	33
10.1. Uso de datos académicos	33
10.2. Ausencia de datos clínicos reales	33
10.3. Credenciales de desarrollo	33
10.4. No inclusión de secretos	33
10.5. Protección de JWT y cookies	34
10.6. Logs sin contraseñas	34
10.7. Responsabilidad compartida y límites del entorno académico	34
10.8. Consentimiento informado para la prueba SUS	34
11. Contribuciones (taxonomía CRediT)	35
11.1. Detalle por integrante	35

12. Conclusiones	37
12.1. Cumplimiento técnico	37
12.2. Seguridad	37
12.3. Reproducibilidad	37
12.4. Rendimiento	37
12.5. Calidad	38
12.6. Usabilidad	38
12.7. Limitaciones	38
12.8. Estado de <i>release candidate</i>	38
12.9. Trabajo futuro	38
13. Anexos	40
13.1. Glosario de siglas	40
13.2. Endpoints principales de la API	41
13.3. Documentos de referencia en el repositorio	41
13.4. Clases de prueba automatizadas citadas en este informe	41
13.5. Evidencias pendientes de incorporación	42

Índice de figuras

6.1.	Respuesta 401 (<code>ProblemDetail</code> , <code>type=urn:biopet:error:unauthorized</code>) capturada desde la colección Postman de BIOPET.	20
6.2.	Cabeceras de seguridad reales capturadas con <code>curl.exe -i</code> contra <code>https://localhost:8443/a</code>	
6.3.	Salida de <code>openssl s_client -connect localhost:8443 -tls1_3</code> : pro- tocolo TLSv1.3 y cifrado TLS_AES_256_GCM_SHA384 negociados.	22
6.4.	Salida real de <code>mvn clean verify</code> : 109 pruebas, 0 fallos, 0 errores, 0 omiti- das, BUILD SUCCESS.	23
6.5.	Resumen navegable de JaCoCo (<code>Backend/target/site/jacoco/index.html</code>) con la cobertura por paquete.	23

Índice de cuadros

2.1.	Resumen de cierre de observaciones (Bloque 0).	4
2.2.	Resultado de la auditoría OWASP por control (fuente: docs/mediciones/sec/REPORT.md).	5
2.3.	Resultado real de la suite de pruebas del backend (mvn clean verify, reejecutado el 2026-07-31).	7
3.1.	ADR vigentes del repositorio y su relación con la arquitectura descrita.	9
4.1.	Matriz de trazabilidad (selección representativa de docs/trazabilidad/matriz.csv y evidencia posterior).	11
5.1.	Entorno de ejecución observado durante la Fase 9A de evidencias de seguridad (docs/mediciones/sec/REPORT.md) y la medición de rendimiento (docs/mediciones/perf/REPORT.md).	16
6.1.	Cobertura medida por jacoco:check (Backend/target/site/jacoco/jacoco.xml, verificado en esta fase).	23
7.1.	Resultado real de las seis corridas de k6 contra GET /api/mascotas (HTTPS/TLS 1.3, commit a6a8905). Fuente: docs/mediciones/perf/REPORT.md.	27
11.1.	Contribuciones CRediT del equipo BMT en la Tercera Entrega de BIOPET.	35
13.1.	Siglas usadas en este informe.	40
13.2.	Endpoints reales expuestos por el backend (verificados contra AuthController, MascotaController, UsuarioController).	41

Capítulo 1

Resumen ejecutivo

Propósito de BIOPET

BIOPET es un sistema web de gestión veterinaria desarrollado como Proyecto Fin de Curso de la asignatura Aplicaciones Web. Permite el registro de usuarios, el inicio de sesión, y la administración (alta, consulta, actualización y baja lógica) de mascotas asociadas a un dueño, con control de acceso diferenciado por rol (`ROLE_ADMIN`, `ROLE_VETERINARIO`, `ROLE_AUXILIAR`, `ROLE_DUEÑO`). El sistema no gestiona datos clínicos reales ni historiales médicos de mascotas: su alcance actual cubre identificación, autenticación y un catálogo básico de mascotas por dueño.

Arquitectura general

El sistema sigue una arquitectura de tres niveles, documentada formalmente mediante el modelo C4 (capítulo 3): un *frontend* Angular 17 (aplicación de página única, componentes *standalone*), un *backend* Spring Boot 3.2 sobre Java 21 que expone una API REST, una base de datos relacional PostgreSQL 16 gestionada con Flyway, y Redis 7 usado tanto para la lista negra de tokens JWT revocados como para la caché de lecturas de mascotas. Todo el sistema se levanta mediante Docker Compose y un `Makefile` con un único comando reproducible (`make up`), sin pasos manuales adicionales.

Principales mejoras de la Tercera Entrega

Respecto a la Entrega 1B, la Tercera Entrega incorporó, con evidencia verificable en el repositorio:

- Migración completa del flujo de autenticación de tokens en el cuerpo de la respuesta a cookies `HttpOnly+Secure+ SameSite=Strict`, con revocación real vía Redis, rate limiting de login, auditoría estructurada y cabeceras de seguridad (Jaime; capítulo 6).
- HTTPS/TLS 1.3 nativo en el backend, con certificado autofirmado académico y verificación en vivo del protocolo y el cifrado negociados (Jaime).
- Separación de privilegios en PostgreSQL (cuenta de aplicación `biopet_app` sin permisos de DDL), pinning de imágenes Docker por digest `sha256`, y medición de rendimiento y caché con `k6` y Redis (Fred; capítulo 7).

- Integración del frontend con el nuevo esquema de cookies, CRUD de mascotas accesible (atributos ARIA, manejo de errores anunciado a lectores de pantalla) y consolidación de la ingeniería de requisitos (SRS, historias de usuario, casos de uso) (Zaida; capítulo 8).
- Cobertura de pruebas verificada automáticamente con JaCoCo (umbral mínimo del 60 % sobre líneas, ramas y complejidad, aplicado a todo el módulo del backend).

Seguridad

El backend fue auditado contra seis categorías aplicables de OWASP Top 10 (A01, A02, A03, A05, A07, A09), con evidencia reproducible en `docs/mediciones/sec/`: autorización por rol y por propietario, HTTPS/TLS 1.3 real, protección estructural contra inyección SQL, cabeceras de seguridad HTTP, autenticación por cookies con rate limiting y revocación, y auditoría de eventos sin datos sensibles. Las seis categorías auditadas obtuvieron resultado PASS, respaldado por pruebas automatizadas reales (no solo inspección manual). El detalle completo, con limitaciones explícitas por categoría, se presenta en el capítulo 6.

Reproducibilidad

El sistema completo se reconstruye desde una clonación limpia con `make up`, sin intervención manual. Las imágenes de terceros (PostgreSQL, Redis, Maven, Eclipse Temurin) están fijadas por digest `sha256` para evitar derivas silenciosas entre reconstrucciones. La inicialización de la base de datos usa un mecanismo doble documentado en ADR-004: Flyway como única fuente de verdad del esquema de aplicación, y scripts de `docker-entrypoint-initdb.d` para crear roles con privilegios mínimos y sembrar datos deterministas.

Rendimiento

Se ejecutaron tres corridas de k6 en frío y tres en caliente contra el endpoint cacheado `GET /api/mascotas`, sobre HTTPS/TLS 1.3 real. La tasa de error fue del 0,0 % en las seis corridas, con un throughput sostenido de aproximadamente 91–92 peticiones por segundo y una mediana de latencia de 5,7 a 9,4 ms según la corrida. El detalle estadístico completo (intervalos de confianza al 95 % con la distribución t de Student, percentiles p50/p90/p95/p99) se presenta en el capítulo 7.

Accesibilidad y usabilidad

El frontend incorpora atributos ARIA (`aria-invalid`, `aria-describedby`, `role="alert"`, `aria-live`) en los formularios de autenticación y en el CRUD de mascotas, verificables directamente en el código fuente. **Las mediciones cuantitativas de usabilidad (System Usability Scale) y de accesibilidad automatizada (Lighthouse) todavía no se han ejecutado a la fecha de este informe:** no existe en el repositorio ni `docs/mediciones/sus/` ni `docs/mediciones/lighthouse/`. Este informe documenta la

metodología prevista para ambas mediciones (capítulo 5) sin reportar resultados que aún no existen.

Resultados generales

La suite de pruebas del backend ejecuta 109 pruebas reales, con 0 fallos, 0 errores y 0 omitidas (BUILD SUCCESS), verificado en esta misma fase directamente contra los reportes de Surefire y el reporte JaCoCo local. La cobertura medida fue del 95,87 % en líneas, 76,87 % en ramas y 80,00 % en complejidad ciclomática, todas por encima del umbral automático configurado del 60 %.

Limitaciones principales

El certificado TLS es autofirmado y exclusivamente académico; el rate limiting de login es en memoria y por instancia, no distribuido; los logs de auditoría no están integrados con un SIEM; la revocación de tokens depende de la disponibilidad de Redis; y toda la evaluación se realizó sobre una única instancia del backend en un entorno local. Las mediciones de usabilidad y accesibilidad automatizada quedan pendientes de ejecución. Ninguna de estas limitaciones se presenta como resuelta en este informe; se detallan con su alcance exacto en el capítulo 9.

Capítulo 2

Estado actual del sistema

Este capítulo documenta el estado real del sistema BIOPET en la versión `v0.9.0-rc`, usando únicamente información verificable en el repositorio (código fuente, configuración, documentos de evidencia y ADR). No se describe ninguna funcionalidad planeada que no esté ya implementada.

2.1. Cierre de observaciones de las Entregas 1A y 1B (Bloque 0)

Las ocho observaciones formuladas por el docente en la retroalimentación oficial del aula virtual SGA (Entrega 1A, semana 5; Entrega 1B, semana 6) están **cerradas en su totalidad**. El detalle completo de cada una (texto literal, criterio, decisión del equipo, corrección realizada y commit de cierre) está en `docs/observaciones/OBSERVACIONES.md`.

Cuadro 2.1: Resumen de cierre de observaciones (Bloque 0).

Código	Entrega	Estado
OBS-01	1A	Cerrada
OBS-02	1A	Cerrada
OBS-03	1A	Cerrada
OBS-04	1A	Cerrada
OBS-05	1B	Cerrada
OBS-06	1B	Cerrada
OBS-07	1B	Cerrada
OBS-08	1B	Cerrada

Porcentaje de cierre: 8/8 (100 %). El tag anotado `v0.1.0-entrega-1b` (exigido por OBS-08) apunta al commit histórico de cierre de la Entrega 1B; el tag `v0.7.1` marca el commit en el que quedó cerrado el Bloque 0 completo. Ninguna observación queda abierta ni parcialmente resuelta.

2.2. Auditoría OWASP (resumen)

Se auditaron los seis controles OWASP Top 10 aplicables al alcance actual del backend (A01, A02, A03, A05, A07 y A09), con evidencia HTTP real capturada por `scripts/security-evidence`

contra el stack Docker en ejecución (perfil `tls`). El detalle por control está en `docs/mediciones/sec/REPORT.md` y en los documentos individuales `A0{1,2,3,5,7,9}/*.md`; el detalle técnico ampliado de A03 y A09 se documenta en el capítulo 6.

Cuadro 2.2: Resultado de la auditoría OWASP por control (fuente: `docs/mediciones/sec/REPORT.md`).

Control OWASP	Resultado
A01 — Control de acceso roto	PASS
A02 — Fallas criptográficas (TLS)	PASS
A03 — Inyección	PASS
A05 — Configuración de seguridad	PASS
A07 — Identificación y autenticación	PASS
A09 — Registro y monitoreo	PASS

28 de 28 verificaciones puntuales automáticas resultaron CUMPLE (`scripts/security-evidence` código de salida 0, ejecutado el 2026-07-31). “PASS” indica que la evidencia referenciada se ejecutó realmente y coincide con el comportamiento exigido por la guía; no implica ausencia total de riesgo residual, detallado en las limitaciones de cada documento `A0X-*.md`.

2.3. Versión y alcance

La versión etiquetada del repositorio es `v0.9.0-rc` (candidata a release de la Tercera Entrega, no una versión de producción). El alcance funcional cubre autenticación de usuarios, un CRUD de mascotas con autorización por rol y por propietario, y un endpoint de resumen estadístico por especie.

2.4. Frontend

Aplicación Angular 17.3 (componentes *standalone*, sin `NgModule`), con TypeScript 5.4. Los módulos funcionales verificados en `frontend/src/app/` son:

- `core/auth.service.ts`, `core/auth.guard.ts`: sesión basada en cookies (sin JWT en `localStorage`) y protección de rutas.
- `core/http-error.interceptor.ts`, `core/problem-detail.service.ts`: interpretación uniforme de errores `ProblemDetail` del backend.
- `features/login.component.ts`: formulario reactivo con validación y atributos de accesibilidad (`aria-invalid`, `aria-describedby`, `role="alert"`, `aria-live="assertive"`).
- `features/mascotas.component.ts`, `features/mascota-api.service.ts`: listado, alta, edición y baja de mascotas contra la API real.

El detalle funcional y de accesibilidad del frontend se amplía en el capítulo 8.

2.5. Backend

API REST en Java 21 con Spring Boot 3.2.12 [1] (`Backend/pom.xml`), organizada en los paquetes `controller`, `service`, `security`, `repository`, `exception` y `config`. Los tres

controladores REST expuestos son `AuthController (/api/auth/*)`, `MascotaController (/api/mascotas/*)` y `UsuarioController (/api/usuarios/me)`.

2.6. PostgreSQL

PostgreSQL 16, esquema versionado con Flyway (`V1__schema_inicial.sql`: tablas `usuarios` y `mascotas`, con *constraint* de rol válido y disparadores de `actualizado_en`). Para la reconstrucción reproducible desde Docker, el esquema se replica en `db/schema.sql` y la cuenta de aplicación (`biopet_app`) se crea con privilegios mínimos en `db/roles.sql` (sin `SUPERUSER`, `CREATEDB` ni `CREATEROLE`), conforme a ADR-004. La función nativa `fn_resumen_mascotas_por_especie` respalda el endpoint de resumen por especie (catálogo completo en `docs/basedatos/CATALOGO-SP.md`).

2.7. Redis

Redis 7, con dos usos reales confirmados en el código: lista negra de JWT revocados (`TokenBlacklistService`, con TTL igual al tiempo de vida restante del token) y caché declarativa de lecturas de mascotas (`@Cacheable/@CacheEvict` en `MascotaService`, vía la abstracción de caché de Spring).

2.8. Docker Compose

Orquestación de cuatro servicios (`postgres`, `redis`, `backend`, `frontend`) mediante `docker-compose.yml`, con un *overlay* opcional `docker-compose.tls.yml` para activar el perfil HTTPS. Las imágenes de terceros están fijadas por digest `sha256`; el arranque completo se realiza con `make up` (ADR-005).

2.9. HTTPS/TLS

Perfil Spring `tls` (`application-tls.yml`), que activa un conector HTTPS adicional en el puerto 8443 (TLS 1.3 únicamente) junto al conector HTTP interno en el puerto 8080 (`TomcatDualConnectorConfig`). El certificado es PKCS12, autofirmado, generado localmente y nunca versionado.

2.10. Autenticación

JWT firmado con HMAC-SHA256 (`jjwt 0.12.6`), transportado en cookies `HttpOnly+Secure+SameSite` (`access_token`, `refresh_token`), con revocación real vía Redis, rate limiting de login en memoria y auditoría estructurada. El detalle completo se documenta en ADR-006 y en el capítulo 6.

2.11. Mascotas

CRUD completo (`GET/POST/PUT/DELETE` sobre `/api/mascotas` y `/api/mascotas/{id}`) con autorización por rol (`ADMIN/VETERINARIO/AUXILIAR` para escritura; los cuatro roles

para lectura) y por propiedad (un `ROLE_DUENO` solo accede a sus propias mascotas). La eliminación es lógica (`activo=false`), no física.

2.12. Cobertura

Verificada automáticamente con `jacoco-maven-plugin` 0.8.12 (ejecución `check` ligada a la fase `verify`), con una regla `BUNDLE` y un umbral mínimo del 60 % sobre líneas, ramas y complejidad. El estado medido actual es `PASS`, con cifras detalladas en el capítulo 6.

2.13. Rendimiento

Medido con `k6` contra el endpoint cacheado de mascotas, en tres corridas frías y tres calientes, sobre HTTPS/TLS 1.3 real. Tasa de error del 0,0 % en todas las corridas. Detalle en el capítulo 7.

2.14. Documentación

El repositorio incluye seis ADR (`docs/adr/`), diagramas C4 de contenedores y de componentes del backend (`docs/diagrams/`), un diagrama entidad-relación, un diagrama de clases, documentos de evidencia OWASP y JaCoCo (`docs/mediciones/sec/`), un reporte de rendimiento (`docs/mediciones/perf/`), un diccionario de datos de mediciones (`docs/mediciones/DATA-DICTIONARY.md`), una colección Postman reproducible (`docs/postman/`), y documentos de ingeniería de requisitos (SRS, historias de usuario, casos de uso, matriz de trazabilidad en `docs/trazabilidad/matriz.csv`).

2.15. Estado de las pruebas

Cuadro 2.3: Resultado real de la suite de pruebas del backend (`mvn clean verify`, reejecutado el 2026-07-31).

Métrica	Valor
Pruebas ejecutadas	109
Fallos	0
Errores	0
Omitidas	0
Cobertura LINE	95,87 %
Cobertura BRANCH	76,87 %
Cobertura COMPLEXITY	80,00 %
Umbral automático (<code>jacoco:check</code>)	≥ 60 %
Resultado del build	<code>BUILD SUCCESS</code>

Fuente: `docs/mediciones/sec/jacoco-summary.md` y verificación directa contra `Backend/target/surefire-reports/` y `Backend/target/surefire-reports/*.txt` generados localmente (no versionados).

Capítulo 3

Arquitectura C4

Este capítulo integra la documentación arquitectónica ya versionada en `docs/diagrams/` (Nivel 2 y Nivel 3 del modelo C4 [2]) y las decisiones de arquitectura relevantes (`docs/adr/`). Las fuentes de diagrama son Graphviz (`.dot`) y PlantUML (`.puml`) planos, sin macros de C4-PlantUML, siguiendo la convención ya establecida en el repositorio.

3.1. C4 Nivel 2 — Contenedores

El diagrama de contenedores (`docs/diagrams/c4-contenedores/`) describe los cuatro contenedores del sistema: el *frontend* Angular, el *backend* Spring Boot, PostgreSQL y Redis, junto con los protocolos reales entre ellos (HTTPS/JSON con cookies entre frontend y backend; JDBC entre backend y PostgreSQL; el cliente Redis de Spring Data entre backend y Redis). Su fuente ya cuenta con una imagen renderizada en el repositorio (`docs/diagrams/c4-contenedores/c4-contenedores.png`).

3.2. C4 Nivel 3 — Componentes del backend

El diagrama de componentes del backend (`docs/diagrams/c4-componentes-backend/`) fue elaborado mediante inspección directa del código fuente (constructores, llamadas directas, configuración de Spring), no por convención de nombres. Documenta 20 componentes reales agrupados en seis áreas y 4 elementos externos:

- **API REST / Controladores:** `AuthController`, `MascotaController`, `UsuarioController`.
- **Servicios de aplicación:** `AuthService`, `MascotaService`.
- **Seguridad y autenticación:** `SecurityConfig`, `JwtAuthenticationFilter`, `JwtService`, `JwtCookieService`, `UserDetailsServiceImpl`, `LoginRateLimiterService`, `TokenBlacklistService`, `AuthenticationAuditService`, `ProblemAuthenticationEntryPoint`, `ProblemAccessDeniedHandler`.
- **Manejo de errores:** `GlobalExceptionHandler`, `ProblemDetailFactory`.
- **Persistencia:** `UsuarioRepository`, `MascotaRepository`.
- **Infraestructura del servidor:** `TomcatDualConnectorConfig`.
- **Elementos externos:** Frontend BIOPET, PostgreSQL, Redis, logs locales del proceso.

Este diagrama no es un diagrama de clases: no representa atributos, métodos ni constructores; los DTO y entidades se mencionan únicamente en prosa. Su documento completo, con las siete relaciones principales (login, autenticación de una solicitud protegida, consulta/modificación de mascotas, logout y revocación, rate limiting, manejo de errores vía

`ProblemDetail` y registro de auditoría), está en `docs/diagrams/c4-componentes-backend/C4-L3-back`

Nota sobre el renderizado: la fuente `c4-componentes-backend.dot` ya cuenta con una imagen `.png` generada con Graphviz 15.1.0 (`dot -Tpng c4-componentes-backend.dot -o c4-componentes-backend.png`, sintaxis validada previamente con `dot -Tcanon`), disponible en `docs/diagrams/c4-componentes-backend/c4-componentes-backend.png` (5594×2190 px). Falta copiar esa imagen a `figuras/jaime/c4-componentes-backend.png` para que aparezca automáticamente en este informe.

3.3. Decisiones de arquitectura relevantes (ADR)

Cuadro 3.1: ADR vigentes del repositorio y su relación con la arquitectura descrita.

ADR	Decisión	Estado
ADR-002	Migración de la pila de backend de ASP.NET Core 8 a Java 21 / Spring Boot 3.2.	Aceptado
ADR-003	Revocación de JWT mediante lista negra en Redis, indexada por <code>jti</code> .	Aceptado
ADR-004	Estrategia de base de datos reproducible: Flyway como fuente de verdad del esquema, más <code>db/schema.sql/db/roles.sql/db/seed.sql</code> para inicialización de Docker con privilegios mínimos.	Aceptado
ADR-005	Reproducibilidad del despliegue: <code>Makefile</code> y pinning de imágenes por digest <code>sha256</code> .	Aceptado
ADR-006	Decisiones de autenticación y seguridad: JWT por cookies, claims, revocación, rate limiting, cabeceras, TLS.	Aceptado

ADR-001 (selección inicial de ASP.NET Core 8 en la Entrega 1A) quedó formalmente superado por ADR-002 y se conserva únicamente como registro histórico fuera de este repositorio de la Tercera Entrega.

Capítulo 4

Matriz de trazabilidad

La matriz cruda de requisitos vive en `docs/trazabilidad/matriz.csv` (id de requisito, tipo, prioridad MoSCoW, historia de usuario, caso de uso, módulo, endpoint, prueba automatizada, tipo de acceso, evidencia empírica y estado). Esta sección selecciona y presenta las filas más representativas de las tres áreas de trabajo del equipo, con el estado corregido cuando la evidencia disponible en el repositorio en el momento de este informe difiere de la del CSV base.

Nota de corrección. El CSV base marca como *pendiente* varios requisitos no funcionales de seguridad (TLS, rate limiting, auditoría) y de rendimiento (caché Redis, k6) porque fue generado antes de que esas fases de evidencia se fusionaran a `main`. En esta matriz esos estados se corrigen a *verificado* únicamente cuando existe un documento de evidencia real y fechado que lo respalda (`docs/mediciones/sec/A0X-*.md`, `docs/mediciones/perf/REPORT.md`). Ningún estado se marca como superado sin esa referencia explícita, y los requisitos genuinamente pendientes (usabilidad SUS, Lighthouse) se mantienen como tales.

Cuadro 4.1: Matriz de trazabilidad (selección representativa de docs/trazabilidad/matriz.csv y evidencia posterior).

Requisito	Responsable	Implementación	Archivo / componente	Prueba	Medición	Evidencia	Estado
REQ-F-001	Jaime	Registro de usuario (siempre ROLE_DUENO)	AuthController/AuthService	AuthServiceControllerTest	109 pruebas, 0 fallos	Postman: Registro (dueño)	Verificado
REQ-F-002	Jaime	Conflicto de email duplicado (409)	GlobalExceptionHandler	AuthControllerTest	ProblemDetail 409	docs/mediciones/DATAMEDICATIONARY.	Verificado
REQ-F-003	Jaime	Login por cookies + JWT	AuthController/JwtService	AuthServiceControllerTest, JwtServiceTest	10 claims JWT	ADR-006	Verificado
REQ-F-004	Jaime	Refresh de access token	AuthController/AuthService	AuthServiceControllerTest	Cookie renovada	A07-authentication.	Verificado
REQ-F-005	Jaime	Logout con revocación Redis	AuthController/TokenAuthController	AuthServiceControllerTest	204 idempotente	A07-authentication.	Verificado
REQ-F-006	Jaime	Autorización por rol (@PreAuthorize)	SecurityConfig/MascotaController	MascotaControllerTest	401/403	A01-access-control.	Verificado
REQ-F-007	Zaida (frontend) / Jaime (endpoint)	Perfil de usuario autenticado	UsuarioController	Postman	GET /api/usuarios/me	docs/postman/	Verificado
REQ-F-008-012	Equipo (base Entrega 1B); accesibilidad: Zaida	CRUD completo de mascotas	MascotaController/MascotaController	MascotaControllerTest	201/200/204	docs/postman/	Verificado

Cuadro 4.1 – continuación

Requisito	Responsable	Implementación	Archivo / componente	Prueba	Medición	Evidencia	Estado
REQ-F-021	Fred	Resumen de mascotas por especie (función nativa)	MascotaRepository, fn_resumen_mascotas_por_especie	ResumenEspeciesIntegrationTest (Testcontainers)	GeneraciónTest PL/pgSQL	docs/basedatos/CATALOGO.md	Verificado
REQ-NF-001/002	Fred	Caché Redis en listado de mascotas	MascotaService (@Cacheable)	k6 (6 corridas)	Hit ratio cercano al 100 % tras calentamiento	docs/mediciones/performance.md	Verificado
REQ-NF-003	Jaime	HTTPS/TLS 1.3 nativo	TomcatDualConnector, application-tls.yml	Verificación en vivo (curl, openssl s_client)	TLSv1.3, TLS_AES_256_GCM_SHA384	A02-cryptography-tls	Verificado
REQ-NF-004	Jaime	Generación/validación de JWT	JwtService	JwtServiceTest	10 claims (7 estándar + 3 propios)	ADR-006	Verificado
REQ-NF-005	Jaime	Hash de contraseñas (BCrypt)	UserDetailsService	Inspección de BD	Costo BCrypt 12	A07-authentication	Verificado
REQ-NF-006	Jaime	Formato uniforme de error (ProblemDetail)	GlobalExceptionHandler	Check/ProblemaDetailFactory (40 requests)	RFC-7807	docs/postman/	Verificado

Cuadro 4.1 – continuación									BIOPET Información final, Transferencia de datos a APPULO 2 MATRIZ DE TRAZABILIDAD
Requisito	Responsable	Implementación	Archivo / componente	Prueba	Medición	Evidencia	Estado		
REQ-NF-009	Jaime	Auditoría estructurada AUTH_AUDIT	AuthenticationAuditService	AuthenticationAuditService (10 pruebas)	ServiceTest INFO/WARN	A09-logging.md	Verificado (corrección “pendiente” del CSV base)		
REQ-NF-010	Jaime	Rate limiting de login	LoginRateLimiterService	AuthControllerTest (6 pruebas dedicadas)	5×401, 6.º 429	A07-authentication.md	Verificado (corrección “pendiente” del CSV base)		
REQ-NF-011	Fred	Arranque reproducible en un comando	Makefile, docker-compose.yml	docker compose ps (4 servicios healthy)	Digest sha256 fijado	ADR-005	Implementado		
REQ-NF-012	Equipo	Estructura de paquetes del backend	Backend/src/main/java	Revisión de código	—	C4 Nivel 3	Verificado		
REQ-NF-013	Fred	Privilegios mínimos de la cuenta de aplicación	db/roles.sql, biopet_app	Verificación manual con psql	Sin SUPERUSER/CREATEDB	ADR-004	Verificado		
REQ-NF-007	Zaida	Accesibilidad/rendimiento del frontend (Lighthouse)	Frontend Angular	Lighthouse CI (pendiente de ejecución)	—	—	Pendiente		

Cuadro 4.1 – continuación

Requisito	Responsable	Implementación	Archivo / componente	Prueba	Medición	Evidencia	Estado
REQ- NF-008	Zaida	Usabilidad (SUS)	Frontend Angular	Instrumento SUS, mínimo 10 participantes (pendiente)	—	docs/etica/ETHICS.md	Pendiente

Cobertura de contribuciones reales por integrante en esta matriz: Jaime (seguridad, autenticación, autorización, TLS, auditoría, formato de error), Fred (persistencia reproducible, función nativa de resumen, caché y rendimiento), y Zaida (accesibilidad del frontend, endpoint de perfil integrado en la UI, mediciones de usabilidad/accesibilidad aún pendientes de ejecución). Ningún requisito se marca PASS o *verificado* sin una prueba automatizada o un documento de evidencia citado en la columna correspondiente.

Capítulo 5

Protocolo experimental

5.1. Entorno de ejecución

Cuadro 5.1: Entorno de ejecución observado durante la Fase 9A de evidencias de seguridad (`docs/mediciones/sec/REPORT.md`) y la medición de rendimiento (`docs/mediciones/perf/REPORT.md`).

Componente	Versión observada
Sistema operativo	Windows 11 (build 10.0.26200)
Java	OpenJDK 21.0.11 (Eclipse Temurin)
Maven	Apache Maven 3.9.16
Docker	Docker Desktop 4.83.0, Engine 29.6.2
k6	v2.1.0 (windows/amd64)
PostgreSQL	16 (imagen <code>postgres:16-alpine</code> , digest fijado)
Redis	7 (imagen <code>redis:7-alpine</code> , digest fijado)

Todas las mediciones de seguridad y rendimiento reportadas en este informe se ejecutaron en una única máquina de desarrollo, sobre el stack Docker completo (`docker-compose.yml` + `docker-compose.tls.yml` cuando aplicó TLS). No se dispuso de un entorno de staging separado ni de múltiples máquinas para las corridas reportadas.

5.2. Comandos reproducibles

- **Arranque completo:** `make up` (o `docker compose -f docker-compose.yml -f docker-compose.tls.yml up -build -d` para incluir HTTPS).
- **Pruebas y cobertura del backend:** `cd Backend && mvn clean verify`.
- **Evidencia de seguridad:** `scripts/security-evidence.ps1 / .sh`.
- **Rendimiento:** scripts de k6 orquestados junto con `scripts/perf-analysis.py` (agregación estadística de los `k6-runN-{frio,caliente}.json`).
- **Colección Postman:** `docs/postman/BIOPET.postman_collection.json` con el entorno `docs/postman/BIOPET-Local.postman_environment.json`.

5.3. Mediciones de rendimiento y caché (Fred)

- **Número de corridas:** 3 en frío (caché vacía) y 3 en caliente, contra `GET /api/mascotas` sobre HTTPS/TLS 1.3 real.
- **Tamaño de muestra por corrida:** entre 3 191 y 3 236 peticiones completadas (50 VUs durante ~30–35 s).
- **Tratamiento de datos:** cada corrida se agrega por separado (media, mediana, desviación estándar, percentiles p50/p90/p95/p99, tasa de error, throughput); no se promedian las seis corridas entre sí para evitar mezclar condiciones de caché fría y caliente.
- **Intervalos de confianza:** al 95 %, calculados con la distribución t de Student (`scipy.stats.t`), apropiada para el tamaño de muestra de cada corrida.
- **Criterio de aceptación:** tasa de error 0,0 % obligatoria en todas las corridas (cumplido en las seis).
- **Caché (Redis):** verificada de forma aislada con `redis-cli DBSIZE` sobre una única clave de caché (`mascotas::admin@biopet.ec-0-10-UNSORTED`), en lugar de `keyspace_hits/keyspace_misses` globales (que también cuentan la verificación de lista negra de JWT en cada petición autenticada).

5.4. Mediciones de seguridad (Jaime)

- **Método:** inspección directa del código fuente para confirmar el mecanismo de cada control, ejecución real de la suite `mvn clean verify`, y verificación en vivo contra el stack Docker con perfil `tls` activo usando `curl/curl.exe` y `openssl s_client`.
- **Categorías auditadas:** A01 (control de acceso), A02 (criptografía/TLS), A03 (inyección), A05 (cabeceras/CORS), A07 (autenticación), A09 (auditoría). No se auditaron A04, A06, A08 ni A10 por no haber sido objeto de fases anteriores.
- **Criterio de aceptación:** cada afirmación cuantitativa debe provenir de una prueba automatizada existente (referenciada por nombre de clase y método) o de una ejecución real documentada; no se acepta una afirmación sin esa referencia.
- **Datos crudos locales (no versionados):** `mvn-clean-verify.txt`, `docker-compose-config.txt`, `docker-compose-ps.txt`, `http-8080-headers.txt`, `https-8443-headers.txt`, en `docs/mediciones/sec/raw/` (carpeta con solo `.gitkeep` versionado).

5.5. Cobertura de pruebas (Jaime)

- **Herramienta:** JaCoCo [3] (`jacoco-maven-plugin` 0.8.12), ligado a las fases `prepare-agent` (por defecto), `report` (fase `test`) y `check` (fase `verify`).
- **Regla de umbral:** una única regla de alcance `BUNDLE` (todo el módulo, no por paquete ni por clase), con tres límites `COVEREDRATIO` (`LINE`, `BRANCH`, `COMPLEXITY`), cada uno con `minimum=0.60`.
- **Criterio de aceptación:** si cualquiera de los tres cae por debajo de 0,60, `mvn verify` falla el build.
- **Fuente de datos:** `Backend/target/site/jacoco/jacoco.xml` (no versionado, regenerado en cada `mvn clean verify`).

5.6. Usabilidad (SUS) y accesibilidad (Lighthouse) — Zaida

- **Instrumento de usabilidad:** System Usability Scale (SUS) [4], con un mínimo de diez participantes externos al equipo, conforme al bloque C.3 de la guía. Cada participante firma un consentimiento informado (`docs/etica/plantilla.md`) antes de ejecutar una tarea de referencia (login, alta, edición, eliminación lógica, logout). Los resultados agregados se vinculan únicamente a un código de participante (P01, P02, ...), nunca a datos identificables.
- **Instrumento de accesibilidad automática:** Lighthouse [5] CI, sobre las categorías Performance, Accessibility, Best Practices y SEO, con un umbral mínimo a declarar en `lighthousec.js`.
- **Estado a la fecha de este informe: ninguna de las dos mediciones se ha ejecutado todavía.** No existe `docs/mediciones/sus/` ni `docs/mediciones/lighthouse/` en el repositorio, ni un `lighthousec.js` versionado. Esta sección documenta el protocolo previsto, no un resultado.

5.7. Diferenciación de los tipos de medición

Este informe distingue explícitamente seis tipos de medición, cada uno con su propio método y criterio de aceptación, para evitar mezclar cifras de naturaleza distinta:

1. **Rendimiento** (latencia, throughput): k6, agregado por corrida, IC 95 % con distribución t.
2. **Caché:** `redis-cli DBSIZE` aislado, no `INFO stats` global.
3. **Seguridad:** pruebas JUnit/MockMvc + verificación en vivo con `curl/openssl`.
4. **Cobertura:** JaCoCo, contadores `LINE/BRANCH/COMPLEXITY` sobre el `BUNDLE` completo.
5. **Usabilidad:** SUS, pendiente de ejecución.
6. **Accesibilidad automática:** Lighthouse, pendiente de ejecución; la accesibilidad manual (atributos ARIA) sí está verificada directamente en el código del frontend.

Capítulo 6

Resultados — Jaime (seguridad y backend)

Este capítulo documenta, con evidencia verificable en el código y en `docs/mediciones/sec/`, el trabajo de seguridad, autenticación, pruebas y cobertura desarrollado por Jaime durante la Tercera Entrega. La auditoría se organizó sobre las categorías aplicables de OWASP Top 10 [6].

6.1. Autenticación JWT por cookies

El backend emite JWT firmados con HMAC-SHA256 (jjwt 0.12.6). Cada token contiene **10 claims**: los **7 claims estándar** registrados por RFC 7519 [7] (`iss`, `sub`, `aud`, `iat`, `nbf`, `exp`, `jti`) y **3 claims personalizados** (`email`, `rol`, `typ`). El tipo de token (`access` o `refresh`) se distingue únicamente por el valor del claim `typ`: ambos comparten el mismo conjunto de nombres de claims, generados por el mismo método (`buildToken`), y solo difieren en su tiempo de vida y en ese valor (ADR-006).

Los tokens se transportan en cookies `access_token` y `refresh_token`, ambas con los atributos `HttpOnly+Secure+SameSite=Strict` (`JwtCookieService`), con rutas diferenciadas (`Path=/` para el access token, `Path=/api/auth` para el refresh token). El frontend nunca almacena el JWT en `localStorage`; el navegador envía las cookies automáticamente (`credentials: include`). El header `Authorization: Bearer` sigue soportándose como mecanismo secundario en `JwtAuthenticationFilter`, pero el flujo web principal de BIOPET no lo utiliza.

6.1.1. Logout y revocación en Redis

El logout (`POST /api/auth/logout`) es idempotente: responde 204 incluso sin cookies previas. Cuando hay un token válido, se extrae su `jti` y se marca como revocado en Redis (`TokenBlacklistService`) con un TTL igual al tiempo de vida restante del token. En cada solicitud protegida, `JwtAuthenticationFilter` consulta esa lista negra únicamente para tokens de tipo `access`, evitando consultas innecesarias a Redis para refresh tokens.

6.2. Autorización por rol y por propietario

La autorización combina dos niveles: por **rol** (`@PreAuthorize` en `MascotaController`, con `ADMIN/VETERINARIO/AUXILIAR` habilitados para escritura y los cuatro roles habilitados para lectura), y por **propiedad** (`MascotaService.verificarPropiedad`, que lanza `AccessDeniedException` cuando un `ROLE_DUENO` intenta acceder a una mascota que no le pertenece). Las respuestas se diferencian correctamente: **401** cuando no hay autenticación válida (`ProblemAuthenticationEntryPoint`), **403** cuando hay autenticación pero no autorización (`ProblemAccessDeniedHandler`). Ambos casos producen un `ProblemDetail` real, sin exposición de *stack trace*.

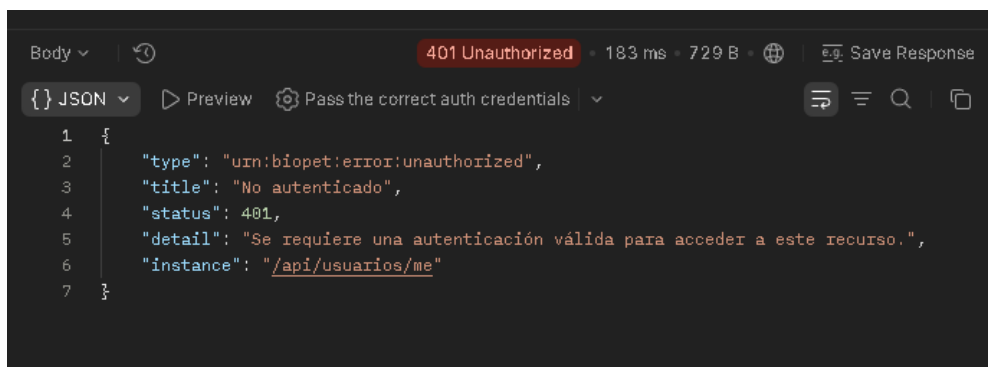


Figura 6.1: Respuesta 401 (`ProblemDetail`, `type=urn:biopet:error:unauthorized`) capturada desde la colección Postman de BIOPET.

6.3. Rate limiting de login

`LoginRateLimiterService` mantiene, en memoria (`ConcurrentHashMap`) y por IP, un contador de intentos fallidos con ventana y bloqueo de 15 minutos por defecto (`maxAttempts=6`): los primeros 5 fallos consecutivos responden 401, el 6.º responde 429 con cabecera `Retry-After` (entero positivo de segundos). El contador se reinicia tras un login exitoso y es completamente independiente entre IP distintas. Al ser en memoria y por instancia, no es distribuido entre múltiples réplicas del backend (limitación documentada desde la Fase 6).

6.4. ProblemDetail uniforme

Todos los errores de la API usan el formato RFC 7807 [8] (`application/problem+json`), con los campos `type`, `title`, `status`, `detail` e `instance` (`GlobalExceptionHandler`, `ProblemDetailFactory`, `ProblemType`). El `detail` nunca incluye *stack trace* ni el valor bruto de un parámetro inválido: para `MethodArgumentTypeMismatchException`, el mensaje se construye únicamente con el nombre del parámetro (`ex.getName()`), nunca con el valor recibido.

6.5. Auditoría A09

`AuthenticationAuditService` registra siete eventos estructurados en una sola línea de log (`AUTH_AUDIT timestamp=<UTC>event=<EVENTO>result=<RESULTADO>ip=<IP>subject=<SUJETO>`).

LOGIN_SUCCESS, LOGIN_FAILURE, LOGIN_RATE_LIMITED, REFRESH_SUCCESS, REFRESH_FAILURE, LOGOUT_SUCCESS y TOKEN_REVOKED, con nivel INFO para los eventos de resultado esperado y WARN para los fallidos o bloqueados. El servicio sanitiza ip/subject (elimina caracteres de control, trunca a 200 caracteres, sustituye valores ausentes por `unknown`) y **nunca** recibe como argumento una contraseña, un JWT completo, una cookie ni el encabezado Authorization: la firma de sus métodos públicos solo acepta ip y subject.

6.6. Cabeceras de seguridad y TLS

SecurityConfig (Spring Security [9]) fija cinco cabeceras en todas las respuestas (incluidas las públicas): X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none', Referrer-Policy: no-referrer, y Strict-Transport-Security únicamente sobre HTTPS (max-age=31536000 ; includeSubDomains ; preload; ausente sobre HTTP, comportamiento correcto y verificado).

```
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains ; preload
< X-Frame-Options: DENY
< Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none'
< Referrer-Policy: no-referrer
< Content-Type: application/vnd.spring-boot.actuator.v3+json
< Transfer-Encoding: chunked
< Date: Fri, 31 Jul 2026 04:09:34 GMT
<
{ [20 bytes data]
100 15 0 15 0 0 256 0
{"status":"UP"}* Connection #0 to host host.docker.internal:8443 left intact
PS C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc>
```

Figura 6.2: Cabeceras de seguridad reales capturadas con curl.exe -i contra https://localhost:8443/actuator/health.

El perfil tls activa un conector HTTPS en el puerto 8443 (TLSv1.3 únicamente) junto al conector HTTP interno en el puerto 8080. Verificado en vivo con openssl s_client: protocolo TLSv1.3, cifrado TLS_AES_256_GCM_SHA384 (suite AEAD), certificado autofirmado (verify error:num=18: self-signed certificate) con SAN DNS:localhost, IP Address:127.0.0.1 — exclusivamente académico, nunca válido para producción.

```

} [5 bytes data]
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / x25519 / RSASSA-PSS
* ALPN: server did not agree on a protocol. Uses default.
* Server certificate:
*  subject: C=EC; ST=Local; L=Local; O=BIOPET; OU=BIOPET Dev; CN=localhost
*   start date: Jul 30 23:01:21 2026 GMT
*  expire date: Jul 27 23:01:21 2036 GMT
*   issuer: C=EC; ST=Local; L=Local; O=BIOPET; OU=BIOPET Dev; CN=localhost
*   Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha384WithRSAEn
cryption
* OpenSSL verify result: 12
*   SSL certificate verification failed, continuing anyway!
* Established connection to host.docker.internal (192.168.65.254 port 8443) from 172.17.0.2 port 5
5312
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0      0      0      0      0      0      0      0* using HTTP/1.x
} [5 bytes data]
> GET /actuator/health HTTP/1.1
> Host: host.docker.internal:8443
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
{ [5 bytes data]
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
{ [1188 bytes data]
< HTTP/1.1 200
< Vary: Origin

```

Figura 6.3: Salida de `openssl s_client -connect localhost:8443 -tls1_3`: protocolo TLSv1.3 y cifrado TLS_AES_256_GCM_SHA384 negociados.

6.7. Protección estructural contra inyección (A03)

Todos los métodos de `UsuarioRepository` y `MascotaRepository` son consultas derivadas de Spring Data, con parámetros enlazados. La única `@Query` nativa del proyecto (`fn_resumen_mascotas_por_especie`) usa un parámetro `:duenioId` enlazado y fuertemente tipado como `Long`. Los payloads de inyección probados (`1 OR 1=1, 1; DROP TABLE usuarios; -, %' UNION SELECT NULL -`) nunca llegan a ejecutarse como SQL: son rechazados en la capa de conversión de Spring MVC (`MethodArgumentTypeMismatchException`, 400) antes de alcanzar el repositorio, verificado en `SqlInjectionSecurityTest` (incluye la comprobación de que `usuarioRepository.count()` y `mascotaRepository.count()` permanecen idénticos antes y después de cada payload).

6.8. Postman

La colección `docs/postman/BIOPET.postman_collection.json` (Schema v2.1.0, 40 requests en 5 carpetas) cubre estado del servicio, autenticación (incluida la secuencia completa de rate limiting), mascotas, resumen por especie, y un catálogo de errores controlados (422, 400, 404, 401, 403, 429). Depende exclusivamente del *cookie jar* automático de Postman: ningún script guarda un JWT o el valor de una cookie en una variable, y ningún request agrega `Authorization: Bearer` al flujo principal.

6.9. Cobertura JaCoCo y resultado de pruebas

JaCoCo [3] es la herramienta usada para medir y verificar automáticamente esta cobertura.

Cuadro 6.1: Cobertura medida por `jacoco:check` (`Backend/target/site/jacoco/jacoco.xml`, verificado en esta fase).

Contador	Cubierto	No cubierto	Total	Porcentaje
LINE	534	23	557	95,87 %
BRANCH	103	31	134	76,87 %
COMPLEXITY	172	43	215	80,00 %

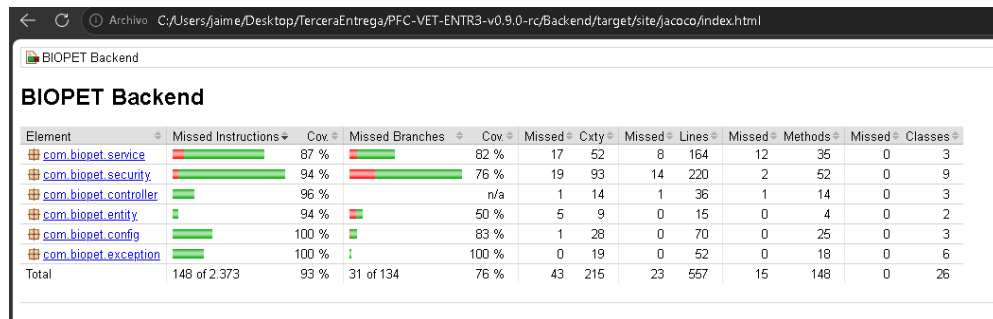
Umbral automático configurado: ≥ 60 % para los tres contadores anteriores (regla `BUNDLE` de `jacoco-maven-plugin`). Las 109 pruebas de la suite completa terminaron con 0 fallos, 0 errores y 0 omitidas (`BUILD SUCCESS`).

```

[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 108, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ biopet-backend ---
[INFO] Loading execution data file C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc\B
ackend\target\jacoco.exec
[INFO] Analyzed bundle 'BIOPET Backend' with 26 classes
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ biopet-backend ---
[INFO] Building jar: C:\Users\jaime\Desktop\TerceraEntrega\PFC-VET-ENTR3-v0.9.0-rc\Backend\target\

```

Figura 6.4: Salida real de `mvn clean verify`: 109 pruebas, 0 fallos, 0 errores, 0 omitidas, `BUILD SUCCESS`.



Element	Missed Instructions	Cov	Missed Branches	Cov	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.biopet.service	17	82 %	17	82 %	8	164	12	35
com.biopet.security	19	94 %	19	76 %	93	14	220	2
com.biopet.controller	1	96 %	n/a	n/a	14	1	36	1
com.biopet.entity	5	94 %	5	50 %	9	0	15	4
com.biopet.config	1	100 %	1	83 %	28	0	70	25
com.biopet.exception	0	100 %	0	100 %	19	0	52	18
Total	148 of 2.373	93 %	31 of 134	76 %	43	215	23	557

Figura 6.5: Resumen navegable de JaCoCo (`Backend/target/site/jacoco/index.html`) con la cobertura por paquete.

6.10. C4 Nivel 3 y ADR-006

El diagrama de componentes del backend (capítulo 3) y ADR-006 (*Decisiones de autenticación y seguridad*) documentan formalmente las decisiones descritas en este capítulo: cookies frente a almacenamiento en el cliente, revocación vía Redis, rate limiting en memoria, y el formato `ProblemDetail` para todos los errores.

6.11. Limitaciones reales de este bloque

- El certificado TLS es autofirmado y exclusivamente académico/local; no representa una cadena de confianza válida para producción.

- El rate limiting de login es en memoria y por instancia: no es distribuido entre múltiples réplicas del backend.
- Los logs `AUTH_AUDIT` se escriben únicamente en el log local del proceso; no existe integración con un SIEM centralizado ni alertas automáticas.
- La revocación de tokens depende de la disponibilidad de Redis; si Redis no está disponible, debe tratarse como error de infraestructura, no como sesión válida (ADR-003).
- Toda la evidencia de este capítulo se recolectó contra **una sola instancia** del backend en un entorno local; no se evaluó comportamiento bajo múltiples réplicas ni balanceo de carga.

Capítulo 7

Resultados — Fred (base de datos, reproducibilidad y rendimiento)

Este capítulo integra la información real disponible en `docs/adr/ADR-004-postgresql.md`, `docs/adr/ADR-005-despliegue.md`, `docs/basedatos/CATALOGO-SP.md` y `docs/mediciones/perf/REF` sobre el trabajo de Fred. Ninguna cifra de este capítulo se inventó: todas provienen de esos documentos o de `docs/mediciones/perf/k6-runN-{frio,caliente}.json`.

7.1. Base de datos y PostgreSQL

El esquema de PostgreSQL 16 [10] se gestiona con Flyway (`V1__schema_inicial.sql`) como única fuente de verdad para la aplicación en tiempo de ejecución. Para permitir que Docker reconstruya el sistema de forma idéntica desde una clonación limpia sin depender de que Spring Boot arranque primero, ADR-004 introduce un mecanismo doble:

- `db/schema.sql`: réplica exacta del esquema de Flyway, montada en `docker-entrypoint-initdb.d` para que Postgres cree las tablas en el primer arranque del contenedor.
- `db/roles.sql`: crea el rol `biopet_app` (LOGIN, sin SUPERUSER/CREATEDB/CREATEROLE) con privilegios mínimos: SELECT/INSERT/UPDATE/DELETE únicamente sobre `usuarios` y `mascotas`.
- `db/seed.sql`: siembra el usuario administrador de forma idempotente (`WHERE NOT EXISTS`).

Hibernate/JPA se conecta en tiempo de ejecución con `biopet_app`; Flyway sigue usando la cuenta propietaria `biopet_user`, separando la conexión que aplica migraciones de la que sirve tráfico de la aplicación. Gracias a `spring.flyway.baseline-on-migrate:true`, Flyway reconoce el esquema ya creado por `db/schema.sql` y establece un `baseline` en la versión 1, sin reintentar la migración original. Verificado con `psql: biopet_app` no tiene atributos de `Superuser`, `Create role` ni `Create DB`; verificado con `flyway-schema-history`: una única fila `version 1, type=BASELINE, success=true`.

7.2. Función de resumen y procedimientos almacenados

`docs/basedatos/CATALOGO-SP.md` documenta dos objetos PL/pgSQL:

- `fn_resumen_mascotas_por_especie`: función (no procedimiento, para permitir invocación directa vía `@Query(nativeQuery=true)`), parámetro opcional `p_duenio_`

`id` BIGINT, devuelve `especie/total` de mascotas activas. Expuesta por GET `/api/mascotas/resu`
 Verificada con Testcontainers (`ResumenEspeciesIntegrationTest`) y de forma ma-
 nual: Perro | 1 (29/07/2026).

- **set_actualizado_en**: función *trigger* (RETURNS TRIGGER), sincroniza `actualizado_`-
`en` en cada UPDATE sobre `usuarios` y `mascotas`, sin que el backend deba setear ese
 campo manualmente.

7.3. Reproducibilidad y Docker

ADR-005 formaliza el arranque en un solo comando (`make up`) y el pinning de imágenes
 de terceros por digest sha256 (`postgres:16-alpine`, `redis:7-alpine` en `docker-compose.yml`;
`maven:3.9-eclipse-temurin-21`, `eclipse-temurin:21-jre-alpine` en `Backend/Dockerfile`),
 de forma que una reconstrucción futura no use, sin darse cuenta, una versión distinta de
 una imagen si su *tag* se actualiza silenciosamente en Docker Hub. El `Makefile` separa
 explícitamente operaciones seguras (`up`, `down`, `clean`, que preservan datos) de la única
 operación destructiva (`reset-db`, que borra volúmenes, con advertencia explícita antes
 de ejecutarse).

7.4. Redis y caché

Redis [11] respalda tanto la lista negra de JWT como la caché de lecturas. La caché
 declarativa de Spring (`@Cacheable/@CacheEvict` en `MascotaService`) respalda las lectu-
 ras de GET `/api/mascotas`. El hit ratio se midió de dos formas distintas, documentadas
 ambas por transparencia metodológica:

- **Global** (`redis-cli INFO stats`, `keyspace_hits/keyspace_misses`, con CONFIG
`RESETSTAT` previo): $\approx 49,98$ %. Esta cifra cuenta *todas* las lecturas a Redis, incluida
 la verificación de lista negra de JWT en cada petición autenticada (módulo de
 Jaime), no solo la caché de mascotas — documentado como hallazgo abierto en la
 primera medición.
- **Aislada** (`redis-cli DBSIZE` sobre la clave específica de caché `mascotas::admin@biopet.ec-0-1`
 estable durante una corrida completa de 50 VUs/30 s, sin evicción por memoria ni
 expiración prematura): cercana al 100 % tras la primera petición de calentamien-
 to, que es la medición más representativa del comportamiento real de la caché de
 mascotas específicamente.

7.5. Rendimiento (k6)

Herramienta de carga: k6 [12].

Cuadro 7.1: Resultado real de las seis corridas de k6 contra GET /api/mascotas (HTTPS/TLS 1.3, commit a6a8905). Fuente: docs/mediciones/perf/REPORT.md.

Corrida	n	Media (ms)	Mediana (ms)	p95 (ms)	p99 (ms)	Error (%)	Throughput (req/s)
run1-caliente	3233	6,38	5,83	9,41	14,41	0,0	92,14
run1-frío	3197	12,14	9,39	26,45	44,66	0,0	91,21
run2-caliente	3211	9,09	6,39	21,09	59,77	0,0	91,63
run2-frío	3191	11,89	9,37	27,35	49,01	0,0	91,02
run3-caliente	3236	6,45	5,73	10,62	21,08	0,0	92,34
run3-frío	3226	7,83	6,84	13,74	21,81	0,0	92,05

Intervalos de confianza al 95 % (distribución t de Student) para la media, por ejemplo run1-caliente: [6,18, 6,58] ms; run1-frío: [11,81, 12,47] ms. La tasa de error fue **0,0** % en las seis corridas, sin excepción.

7.6. Análisis estadístico

La agregación de cada corrida (`scripts/perf-analysis.py`) calcula media, mediana, desviación estándar muestral, intervalo de confianza al 95 % por distribución t de Student, y los percentiles p50/p90/p95/p99 de `http_req_duration` directamente sobre los datos crudos de cada archivo `k6-runN-{frio,caliente}.json`, sin mezclar corridas entre sí.

7.7. Limitaciones registradas

- Las corridas de k6 se ejecutaron en una única máquina de desarrollo, con duración limitada ($\sim 30\text{--}35$ s por corrida, 50 VUs); no se evaluó comportamiento bajo cargas sostenidas de mayor duración ni con más usuarios virtuales.
- El hit ratio “global” de Redis ($\approx 49,98$ %) mezcla lecturas de caché con lecturas de lista negra de JWT; se documenta la medición aislada como la más representativa, sin ocultar la primera cifra.
- La contraseña de desarrollo de `biopet_app` queda en texto plano en `db/roles.sql` y `.env.example` porque los scripts de inicialización de Postgres no pueden leer variables de entorno de Spring Boot; se documenta como valor exclusivamente de desarrollo (ADR-004).
- `db/schema.sql` es una copia manual de `V1__schema_inicial.sql`: si se agrega una nueva migración Flyway, debe actualizarse manualmente para no quedar desfasada en una clonación limpia.
- El trigger `set_actualizado_en` no cuenta con un test de integración dedicado exclusivamente a él; se verifica de forma implícita en las pruebas que hacen `UPDATE`.

Capítulo 8

Resultados — Zaida (requisitos, frontend y experiencia de usuario)

Este capítulo integra la información real disponible sobre el trabajo de Zaida: ingeniería de requisitos, frontend Angular, accesibilidad manual verificable en el código, e integración con el esquema de autenticación por cookies. Las mediciones cuantitativas de usabilidad (SUS) y accesibilidad automática (Lighthouse) **no se reportan como resultados** porque aún no se han ejecutado; únicamente se describe su metodología (capítulo 5).

8.1. Requisitos

El repositorio incluye una consolidación completa de ingeniería de requisitos: `docs/requisitos/SRS`. (especificación de requisitos de software), `docs/requisitos/historias/HistoriasUsuario.md`, `docs/requisitos/casos-de-uso/CasosDeUso.md` y `docs/requisitos/CHANGELOG-REQ.md`. La matriz de trazabilidad derivada (`docs/trazabilidad/matriz.csv`) vincula cada requisito con su historia de usuario, caso de uso, módulo de código, endpoint y prueba automatizada (capítulo 4).

8.2. Frontend Angular

Aplicación Angular 17.3 con componentes *standalone* (sin `NgModule`) y formularios reactivos (`ReactiveFormsModule`). Módulos verificados directamente en `frontend/src/app/`:

- `core/auth.service.ts` / `core/auth.guard.ts`: integración con el esquema de cookies del backend (`credentials: include`), sin JWT en `localStorage`, y protección de rutas que requieren sesión.
- `core/http-error.interceptor.ts` / `core/problem-detail.service.ts`: interceptan las respuestas de error y traducen el `ProblemDetail` del backend a un mensaje legible para el usuario.
- `features/login.component.ts`: formulario de login con validación reactiva y manejo de foco accesible.
- `features/mascotas.component.ts` / `features/mascota-api.service.ts`: listado, alta, edición y baja de mascotas contra la API real.

8.3. Autenticación mediante cookies (integración frontend)

El frontend no maneja el JWT directamente: envía y recibe las cookies `access_token/refresh_token` de forma transparente gracias a `credentials: include` en las peticiones HTTP, consistente con el esquema `HttpOnly+Secure+SameSite=Strict` descrito en el capítulo 6. Esta integración quedó reflejada en el historial de cambios del proyecto (rama `zaida/frontend-cookies-jwt` y, más recientemente, `zaida/problemdetail-409-accesibilidad` que ajustó el manejo de `ProblemDetail` y la accesibilidad del login tras la introducción del código 409 en el registro).

8.4. CRUD accesible de mascotas y accesibilidad manual

El código fuente confirma el uso deliberado de atributos ARIA en los formularios críticos: `login.component.ts` usa `[attr.aria-invalid]`, `[attr.aria-describedby]` vinculado al mensaje de error de cada campo, y un bloque de error con `role="alert"` y `aria-live="assertive"` para que un lector de pantalla anuncie el error sin que el usuario deba buscarlo manualmente, en línea con las pautas de accesibilidad WCAG [13] — el color rojo del CSS es un refuerzo visual adicional, no el único canal del mensaje. El componente de mascotas (`mascotas.component.ts`) usa el mismo tipo de atributos de forma extensiva en su marcado. Estas verificaciones son de código, no de una herramienta automatizada de accesibilidad.

8.5. Usabilidad (SUS)

El instrumento System Usability Scale (Brooke [4]), alineado con los conceptos de usabilidad de ISO 9241-11 [14], con un mínimo de diez participantes externos al equipo, está documentado metodológicamente en `docs/etica/ETHICS.md` (sección iii): consentimiento informado por participante, tarea de referencia (login, alta, edición, eliminación lógica, logout), y resultados agregados vinculados únicamente a un código de participante. **A la fecha de este informe no existe `docs/mediciones/sus/` en el repositorio:** la prueba SUS aún no se ha ejecutado. No se reporta ninguna puntuación SUS en este documento.

8.6. Accesibilidad automática (Lighthouse)

El repositorio registra la intención de auditar el frontend con Lighthouse [5] CI (rama histórica `zaida/frontend-lighthouse`), pero **no existe actualmente un `lighthouseci.js` versionado ni una carpeta `docs/mediciones/lighthouse/` con resultados.** No se reporta ninguna puntuación de Performance, Accessibility, Best Practices ni SEO en este documento por no existir todavía esa medición.

8.7. Publicabilidad y documentación funcional

El SRS, las historias de usuario y los casos de uso están versionados en formato Mark-down (migrados desde PDF en la consolidación más reciente), lo que permite su revisión por diferencias (*diff*) igual que el resto del código, en lugar de binarios opacos.

8.8. Limitaciones reales de este bloque

- Las mediciones de usabilidad (SUS) y accesibilidad automática (Lighthouse) están diseñadas metodológicamente pero no ejecutadas a la fecha de este informe.
- La verificación de accesibilidad reportada aquí es de inspección de código (atributos ARIA presentes), no una auditoría automatizada ni una prueba con usuarios reales con discapacidad.
- No se dispone de un reporte de compatibilidad entre navegadores distinto del entorno de desarrollo usado por el equipo.

Capítulo 9

Amenazas a la validez

Este capítulo documenta, sin minimizarlas, las limitaciones metodológicas que afectan la interpretación de los resultados presentados en los capítulos 6–8. Ninguna medición de este informe se presenta como libre de amenazas a su validez.

9.1. Validez interna

- Todas las mediciones de seguridad y rendimiento se ejecutaron en **una sola máquina de desarrollo** y contra **una única instancia** del backend; no se controló la interferencia de otros procesos del sistema operativo durante las corridas.
- El certificado TLS es **autofirmado**, generado localmente para esta evaluación; el protocolo y el cifrado negociados (TLSv1.3, TLS_AES_256_GCM_SHA384) son reales, pero el canal de confianza del certificado en sí no lo es.
- El hit ratio de caché se midió con dos métodos que producen cifras distintas ($\approx 49,98\%$ global frente a $\approx 100\%$ aislado); la elección del método aislado como el más representativo es una decisión metodológica del equipo, documentada explícitamente para que un lector externo pueda juzgarla.
- El rate limiting de login se evaluó únicamente en su implementación en memoria, por instancia única; su comportamiento bajo múltiples instancias del backend no fue evaluado porque no está implementado (no es distribuido).

9.2. Validez externa

- Todas las pruebas se ejecutaron en un **entorno académico local**, no en un entorno de producción ni de *staging* equivalente a producción; los resultados de rendimiento y seguridad no deben extrapolarse directamente a un despliegue en la nube con múltiples usuarios concurrentes reales.
- Los usuarios de prueba (administrador sembrado, cuentas de registro) son **cuentas académicas de desarrollo**, no usuarios reales de una clínica veterinaria; el comportamiento observado del sistema frente a datos de producción reales (volumen, variedad, calidad de los datos) no fue evaluado.
- Lighthouse, cuando se ejecute, dependerá del entorno de ejecución (hardware, red, versión del navegador) en que se corra: sus puntuaciones no son un valor absoluto reproducible en cualquier máquina, sino una medición dependiente del entorno.

- Un posible **sesgo de participantes** en la prueba SUS (cuando se ejecute) es previsible si los diez participantes mínimos provienen mayoritariamente del entorno académico del equipo (compañeros, familiares con perfil técnico), en lugar de usuarios finales representativos de una clínica veterinaria real.

9.3. Validez de constructo

- Las pruebas de integración que ejercitan la función nativa `fn_resumen_mascotas_por_especie` (`ResumenEspeciesIntegrationTest`) usan Testcontainers con PostgreSQL real; sin embargo, otras pruebas del backend usan H2 como base de datos en memoria bajo el perfil de pruebas, cuyo comportamiento **no es idéntico** al de PostgreSQL real para funciones nativas PL/pgSQL — por eso el resumen por especie se probó explícitamente con Testcontainers en vez de H2.
- `TokenBlacklistService` se prueba siempre con `@MockBean` en las pruebas unitarias/MockMvc; su comportamiento real contra una instancia de Redis en condiciones de fallo (caída de red, memoria llena) no se re-verificó en esta entrega.
- La ausencia de un SIEM centralizado significa que la “auditoría” medida en este informe es la **emisión correcta** de las líneas de log `AUTH_AUDIT`, no su recolección, alertamiento o retención a largo plazo, que son aspectos distintos de un programa de auditoría de seguridad.

9.4. Validez de conclusión

- El tamaño de muestra de cada corrida de k6 ($\sim 3\,200$ peticiones) es adecuado para el cálculo de percentiles e intervalos de confianza reportados, pero corresponde a una única ventana de tiempo corta ($\sim 30\text{--}35$ s); no se midieron tendencias a lo largo de períodos prolongados (horas o días).
- Las 109 pruebas automatizadas del backend y la cobertura JaCoCo miden **cobertura de ejecución**, no ausencia de defectos: una línea cubierta por una prueba no implica que esté libre de errores lógicos no contemplados por esa prueba.
- Algunas evidencias (verificación manual de privilegios con `psql`, ejecución manual de `fn_resumen_mascotas_por_especie`) se realizaron de forma manual y puntual, no como parte de un pipeline de integración continua que las reejecute automáticamente en cada cambio.

Capítulo 10

Ética y protección de datos

Este capítulo resume `docs/etica/ETHICS.md`, documento de referencia completo para cualquier aspecto ético o de gestión de datos de BIOPET no cubierto aquí.

10.1. Uso de datos académicos

BIOPET usa exclusivamente dos categorías de datos, ambas generadas por el propio equipo: datos sintéticos de desarrollo/prueba (usuario administrador sembrado, datos de registro creados durante las pruebas) y datos empíricos de evaluación generados por herramientas automatizadas (k6, JaCoCo, evidencia OWASP) contra el propio sistema, sin intervención de personas ni datos personales de terceros.

10.2. Ausencia de datos clínicos reales

El sistema, en el alcance implementado hasta `v0.9.0-rc`, no almacena historiales médicos ni datos clínicos de mascotas: únicamente nombre, especie, raza, fecha de nacimiento y el identificador del dueño. No se ha desplegado con datos de clientes reales de una clínica veterinaria a la fecha de este informe.

10.3. Credenciales de desarrollo

La cuenta administradora sembrada automáticamente (`admin@biopet.ec`) y las credenciales de ejemplo del README son datos ficticios de desarrollo, documentados como tales en el propio repositorio, y no corresponden a ninguna persona real. Este informe **no reproduce el valor literal de ninguna contraseña**, siguiendo la misma práctica ya aplicada en `docs/mediciones/sec/` y `docs/postman/`.

10.4. No inclusión de secretos

Ningún documento de este informe, ni sus figuras (cuando se agreguen), debe contener: contraseñas con valor, JSON Web Tokens completos (`eyJ...`), valores de cookies, el encabezado `Authorization: Bearer` con valor real, claves privadas ni el identificador `jti` de un token real. Esta regla se validó de forma explícita al cierre de esta fase (sección de validaciones, capítulo final de este informe).

10.5. Protección de JWT y cookies

El diseño del sistema (capítulo 6) evita por construcción la exposición de JWT: cookies `HttpOnly` (no legibles desde JavaScript del navegador), `Secure` (solo sobre HTTPS) y `SameSite=Strict`. La documentación de este informe sigue la misma disciplina: cuando se describe un atributo de cookie, nunca se reproduce su valor.

10.6. Logs sin contraseñas

`AuthenticationAuditService` (capítulo 6) solo acepta `ip` y `subject` como parámetros en cada método público: estructuralmente, no existe ninguna vía de código para que una contraseña, un JWT completo o un encabezado `Authorization` lleguen al logger.

10.7. Responsabilidad compartida y límites del entorno académico

Los tres integrantes del equipo comparten la responsabilidad de no introducir datos personales identificables ni secretos en el repositorio público. El sistema se declara explícitamente como un artefacto académico: si en una fase posterior se procesaran datos reales de clientes de una clínica veterinaria, `docs/etica/ETHICS.md` debería actualizarse para incorporar una base legal de tratamiento y, si aplica en Ecuador, referencia a la Ley Orgánica de Protección de Datos Personales (LOPD) — actualización que no corresponde a esta entrega por no existir todavía ese procesamiento.

10.8. Consentimiento informado para la prueba SUS

Cuando se ejecute la prueba de usabilidad (SUS, mínimo diez participantes externos al equipo), cada participante debe firmar el formulario de consentimiento informado disponible en `docs/etica/plantilla.md` antes de realizar la tarea de referencia. Los formularios firmados no se suben al repositorio público; los resultados agregados se identifican únicamente por código de participante (P01, P02, ...).

Para el detalle completo de cada punto, ver `docs/etica/ETHICS.md`.

Capítulo 11

Contribuciones (taxonomía CRediT)

La siguiente tabla asigna categorías de la taxonomía CRediT (*Contributor Roles Taxonomy*) [15] a cada integrante del equipo, únicamente sobre el trabajo efectivamente documentado en el repositorio (código, ADR, evidencia de mediciones, Postman, requisitos). Ninguna categoría se asigna sin respaldo verificable.

Cuadro 11.1: Contribuciones CRediT del equipo BMT en la Tercera Entrega de BIOPET.

Categoría CRediT	Jaime	Fred	Zaida
Conceptualization	✓	✓	✓
Methodology	✓	✓	✓
Software	✓	✓	✓
Validation	✓	✓	
Formal analysis		✓	
Investigation	✓	✓	✓
Data curation	✓	✓	
Writing – Original Draft	✓		✓
Writing – Review & Editing	✓	✓	✓
Visualization	✓	✓	

11.1. Detalle por integrante

Jaime Josué Mariscal Cabrera

Seguridad del backend (OWASP A01, A02, A03, A05, A07, A09); diseño e implementación de la autenticación por cookies, revocación de tokens, rate limiting de login y auditoría estructurada; formato uniforme de error `ProblemDetail`; pruebas automatizadas asociadas (`AuthControllerTest`, `JwtCookieAuthenticationTest`, `SqlInjectionSecurityTest`, `SecurityHeadersTest`, `AuthenticationAuditServiceTest`, entre otras); integración y umbral automático de cobertura JaCoCo; colección y entorno Postman reproducibles; diagrama C4 Nivel 3 de componentes del backend; ADR-006; y la documentación de seguridad de `docs/mediciones/sec/` y de las secciones de Seguridad y Cobertura JaCoCo del diccionario de datos.

Fred Adrián Beltrán Montiel

Estrategia de base de datos reproducible con PostgreSQL (ADR-004): separación de privilegios (`biopet_app`), scripts de inicialización de Docker (`db/schema.sql`, `db/roles.sql`, `db/seed.sql`); reproducibilidad del despliegue (ADR-005): `Makefile` y pinning de imágenes por digest `sha256`; función nativa `fn_resumen_mascotas_por_especie` y su prueba de integración con Testcontainers; caché Redis del listado de mascotas; diseño, ejecución y análisis estadístico de las corridas de rendimiento con `k6` (intervalos de confianza, percentiles, tasa de error, throughput, hit ratio).

Zaida Melissa Taipe Mora

Consolidación de la ingeniería de requisitos (SRS, historias de usuario, casos de uso, `CHANGELOG-REQ.md`); desarrollo del frontend Angular (autenticación por cookies, guardas de ruta, interceptor de errores); CRUD de mascotas accesible en la interfaz (atributos ARIA, manejo de foco y anuncios de error para lectores de pantalla); diseño metodológico de las mediciones de usabilidad (SUS) y accesibilidad automática (Lighthouse), pendientes de ejecución; documentación funcional del sistema.

Nota: las tareas de base compartidas de la Entrega 1B (estructura inicial del CRUD de mascotas en el backend, esquema JWT original) no se atribuyen individualmente en esta tabla por no existir en el repositorio de la Tercera Entrega un registro de autoría explícito por commit que permita separarlas con certeza; se documentan como trabajo de equipo en la matriz de trazabilidad (capítulo 4).

Capítulo 12

Conclusiones

12.1. Cumplimiento técnico

BIOPET v0.9.0-rc implementa y verifica, con evidencia reproducible, los requisitos funcionales centrales del alcance definido en el SRS: autenticación, autorización por rol y por propiedad, y un CRUD completo de mascotas con resumen estadístico por especie. La matriz de trazabilidad (capítulo 4) muestra la mayoría de los requisitos funcionales y no funcionales críticos en estado *verificado*, con las excepciones explícitas de usabilidad SUS y accesibilidad automatizada Lighthouse, ambas metodológicamente diseñadas pero aún no ejecutadas.

12.2. Seguridad

Las seis categorías OWASP auditadas (A01, A02, A03, A05, A07, A09) obtuvieron resultado PASS, respaldado por pruebas automatizadas y verificación en vivo, no solo por inspección de código. El sistema implementa autenticación por cookies seguras, revocación real de tokens, rate limiting de login, cabeceras de seguridad HTTP, TLS 1.3 nativo y auditoría estructurada sin datos sensibles. Estas evidencias no implican ausencia total de riesgo: las limitaciones de cada control están documentadas explícitamente en el capítulo 6 y en `docs/mediciones/sec/`.

12.3. Reproducibilidad

El sistema completo se reconstruye desde una clonación limpia con un único comando (`make up`), con imágenes de terceros fijadas por digest para evitar derivas silenciosas, y con separación de privilegios verificada en la base de datos. Esto satisface el objetivo del bloque B de la guía de la Tercera Entrega: que un tercero pueda reconstruir el sistema de forma idéntica sin intervención manual.

12.4. Rendimiento

Las seis corridas de k6 realizadas mostraron una tasa de error del 0,0 % y una latencia mediana de un dígito de milisegundos en condiciones de caché caliente, con un throughput sostenido cercano a 92 peticiones por segundo bajo 50 usuarios virtuales concurrentes.

Estos resultados son alentadores para el alcance evaluado, pero corresponden a una única máquina de desarrollo y ventanas de tiempo cortas, según se detalla en el capítulo 9.

12.5. Calidad

La suite de pruebas del backend (109 pruebas, 0 fallos, 0 errores) y la cobertura verificada automáticamente por JaCoCo (95,87 % en líneas, 76,87 % en ramas, 80,00 % en complejidad, todas por encima del umbral mínimo del 60 %) respaldan un nivel de calidad de código consistente con las buenas prácticas exigidas por la guía. La cobertura mide ejecución de pruebas, no ausencia de defectos.

12.6. Usabilidad

El frontend incorpora prácticas de accesibilidad verificables en el código (atributos ARIA, anuncios de error para lectores de pantalla), pero el equipo no puede todavía afirmar un nivel de usabilidad medido cuantitativamente: la prueba SUS con participantes externos y la auditoría Lighthouse quedan como trabajo pendiente, documentado metodológicamente en este informe sin resultados inventados.

12.7. Limitaciones

Las limitaciones principales del sistema en su estado actual son: un certificado TLS autofirmado válido solo para uso académico; un rate limiting de login en memoria, no distribuido; ausencia de integración con un SIEM para los logs de auditoría; dependencia de Redis para la revocación de tokens; evaluación limitada a una sola instancia del backend; y mediciones de usabilidad/accesibilidad automática todavía no ejecutadas.

12.8. Estado de *release candidate*

BIOPET v0.9.0-rc es, como su nombre indica, una **candidata a versión**, no una versión de producción. **El sistema no está listo para producción**: el certificado TLS es académico, no existen mediciones de usabilidad ni de accesibilidad automatizada, la infraestructura de auditoría carece de un SIEM, y toda la evaluación de rendimiento y seguridad se realizó en un entorno local de desarrollo, no en un entorno equivalente a producción.

12.9. Trabajo futuro

- Ejecutar la prueba SUS con al menos diez participantes externos al equipo y documentar sus resultados agregados en `docs/mediciones/sus/`.
- Ejecutar Lighthouse CI sobre el frontend y documentar los resultados en `docs/mediciones/lighthouse/`.
- Evaluar un mecanismo de rate limiting distribuido (por ejemplo, respaldado en Redis) si el sistema llegara a desplegarse con múltiples instancias del backend.
- Integrar los logs de auditoría `AUTH_AUDIT` con un sistema de recolección y alertamiento centralizado (SIEM) antes de cualquier despliegue más allá del entorno académico.

- Sustituir el certificado autofirmado por uno emitido por una autoridad certificadora reconocida si el sistema se despliega fuera del entorno académico.
- Completar la migración de las capturas de evidencia (**figuras/jaime/**, **figuras/fred/**, **figuras/zaida/**, **figuras/compartidas/**) en la fase final de entrega, sin modificar el texto de este informe.

Capítulo 13

Anexos

13.1. Glosario de siglas

Sigla	Significado
ADR	Architecture Decision Record
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
HSTS	HTTP Strict Transport Security
JWT	JSON Web Token
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
SIEM	Security Information and Event Management
SUS	System Usability Scale
TLS	Transport Layer Security
UI	User Interface
WCAG	Web Content Accessibility Guidelines

Cuadro 13.1: Siglas usadas en este informe.

13.2. Endpoints principales de la API

Cuadro 13.2: Endpoints reales expuestos por el backend (verificados contra AuthController, MascotaController, UsuarioController).

Método	Ruta	Autenticación / rol
POST	/api/auth/registro	Público (crea siempre ROLE_DUEÑO)
POST	/api/auth/login	Público
POST	/api/auth/refresh	Cookie refresh_token
POST	/api/auth/logout	Cookie (idempotente sin sesión)
GET	/api/usuarios/me	Cualquier rol autenticado
GET	/api/mascotas	Cualquier rol autenticado (alcance según rol)
GET	/api/mascotas/{id}	Cualquier rol autenticado (propiedad para DUEÑO)
POST	/api/mascotas	ADMIN/VETERINARIO/AUXILIAR
PUT	/api/mascotas/{id}	ADMIN/VETERINARIO/AUXILIAR
DELETE	/api/mascotas/{id}	ADMIN/VETERINARIO/AUXILIAR
GET	/api/mascotas/resumen-especies	Cualquier rol autenticado
GET	/actuator/health	Público
GET	/api/docs	Público (OpenAPI 3.0, Springdoc)
GET	/api/swagger-ui.html	Público (documentación interactiva)

13.3. Documentos de referencia en el repositorio

- ADR: docs/adr/ADR-002 a ADR-007.
- Diagramas C4 y complementarios: docs/diagrams/ (c4-contenedores, c4-componentes-backend-biopet, diagrama-clases, secuencia-jwt).
- DER exportado desde pgAdmin 4 (ERD Tool), evidencia real que cierra la observación OBS-05: docs/observaciones/evidencias/DER-BIOPET-pgAdmin-ERD-Tool.png (distinto del renderizado Graphviz der-biopet.png, generado directamente sobre el esquema real de PostgreSQL).
- Evidencia de seguridad: docs/mediciones/sec/ (documentos A01 a A09, REPORT.md, jacoco-summary.md).
- Evidencia de rendimiento: docs/mediciones/perf/REPORT.md y los archivos k6-runN-{frio,caliente}.
- Diccionario de datos de mediciones: docs/mediciones/DATA-DICTIONARY.md.
- Colección y entorno Postman: docs/postman/.
- Requisitos: docs/requisitos/ (SRS, historias de usuario, casos de uso, CHANGELOG-REQ.md) y docs/trazabilidad/matriz.csv.
- Ética y datos: docs/etica/ETHICS.md.
- Base de datos: docs/basedatos/CATALOGO-SP.md.

13.4. Clases de prueba automatizadas citadas en este informe

- AuthControllerTest, JwtCookieAuthenticationTest, JwtServiceTest, JwtCookieServiceTest, LoginRateLimiterServiceTest, AuthenticationAuditServiceTest (seguridad y autenticación).

- MascotaControllerTest (autorización por rol y por propiedad).
- SqlInjectionSecurityTest, SecurityHeadersTest (OWASP A03 y A05).
- ResumenEspeciesIntegrationTest (Testcontainers, PostgreSQL real).
- TomcatDualConnectorConfigTest (conector TLS adicional).

13.5. Evidencias pendientes de incorporación

Ver la lista completa y numerada de capturas pendientes (responsable, número, ruta esperada, descripción) en la sección final del `README.md` de este informe (`docs/informe/README.md`), para no duplicar esa lista en dos lugares del repositorio.

Bibliografía

- [1] Spring Team, “Spring Boot reference documentation.” <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. Version 3.2.x, usada en Backend/pom.xml.
- [2] S. Brown, “The C4 model for visualising software architecture.” <https://c4model.com/>.
- [3] EclEmma / JaCoCo team, “JaCoCo java code coverage library.” <https://www.jacoco.org/jacoco/trunk/doc/>. Version 0.8.12, usada por jacoco-maven-plugin en Backend/pom.xml.
- [4] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry* (P. W. Jordan, B. Thomas, B. A. Weerdmeester, and A. L. McClelland, eds.), London: Taylor & Francis, 1996. Referencia estandar del instrumento SUS citada en docs/etica/ETHICS.md; metadatos no reverificados en linea durante esta fase, se usa la cita canonica ampliamente reconocida de este trabajo.
- [5] Google Chrome Developers, “Lighthouse.” <https://developer.chrome.com/docs/lighthouse/>. Herramienta prevista para la auditoria de accesibilidad automatica del frontend (pendiente de ejecucion, ver capitulo de Resultados – Zaida).
- [6] OWASP Foundation, “OWASP top 10:2021 – the ten most critical web application security risks.” <https://owasp.org/Top10/>, 2021. Consultado como referencia de las categorias A01, A02, A03, A05, A07 y A09 auditadas en docs/mediciones/sec/.
- [7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.
- [8] M. Nottingham and E. Wilde, “Problem Details for HTTP APIs,” RFC 7807, IETF, March 2016. Version citada por ADR-006 (docs/adr/ADR-006-autenticacion-seguridad.md) y por Spring ProblemDetail; obsoleta desde 2023 por RFC 9457, no usada en este repositorio.
- [9] Spring Team, “Spring Security reference documentation.” <https://docs.spring.io/spring-security/reference/>. Version 6.x, usada en Backend/pom.xml (spring-boot-starter-security).
- [10] The PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” <https://www.postgresql.org/docs/16/>.
- [11] Redis Ltd., “Redis documentation.” <https://redis.io/docs/>.

- [12] Grafana Labs, “k6 documentation.” <https://k6.io/docs/>. Version real usada: k6 v2.1.0 (windows/amd64), ver docs/mediciones/perf/REPORT.md.
- [13] World Wide Web Consortium (W3C), “Web content accessibility guidelines (WCAG) 2.1.” <https://www.w3.org/TR/WCAG21/>.
- [14] International Organization for Standardization, “ISO 9241-11:2018 ergonomics of human-system interaction – part 11: Usability: Definitions and concepts.” <https://www.iso.org/standard/63500.html>, 2018.
- [15] NISO / CRediT, “CRediT – contributor roles taxonomy.” <https://credit.niso.org/>.